Define the Problem Statements

Project: iMovies MERN **Document:** Define the Problem Statements

1. Background

Movie ticket booking can involve scattered information about movies, show times, prices, and seat availability. A small, easy-to-use web application can bring these actions into one workflow.

2. Primary problem statement

How might we provide moviegoers with a straightforward web experience to browse movies, select a show, choose seats, create a booking, and manage their bookings while allowing an administrator to maintain movie listings?

3. User problem statements

- Moviegoers need movie details in one place so they can decide what to watch.
- Registered users need a simple booking process that provides a clear seat and price summary.
- Users need to see their own bookings and cancel a booking when permitted.
- Administrators need authenticated tools to add or remove movies and review bookings.
- The system needs to reduce accidental duplicate seat reservations and protect private user data.

4. Why it matters

A clear workflow reduces confusion, keeps movie information organized, and makes booking records easier to manage. Correct validation and authorization are essential for trust.

5. Target users

User group	Needs
Visitor / moviegoer	Browse and search movie listings; understand available information.
Registered user	Sign in, book seats, view personal bookings and cancel where allowed.
Administrator	Maintain movie listings and inspect booking records securely.

6. Scope

In scope: user registration/login, movie listing/search, show selection, seat selection, booking creation, demo-payment flow if enabled, My Bookings/cancellation, admin login, movie management and booking review. Out of scope for this prototype: real payment processing, real cinema inventory integrations, and production-grade multi-cinema management.

7. Success criteria

- A new user can register and sign in using valid details.
- Movies load from the backend database.
- A signed-in user can submit a booking with a selected movie, show and seats.
- The server validates bookings and prevents duplicate seat reservations where its reservation logic is active.

- Users can view only their own bookings and cancel their own eligible bookings.
- Admin-only endpoints reject requests without valid admin authorization.

8. Constraints

The prototype depends on a running Node.js backend, MongoDB database and React/Vite frontend. Availability and booking correctness depend on server-side validation and the reservation index being initialized successfully.

Prepared for academic project documentation. Verify implementation-specific details and fill in actual test evidence before final submission.